

Dimensionality Reduction for High-Dimensional Data: PCA, Kernel PCA and t-SNE on MNIST, from Notebook to Monitored Service

Xaysomvang (Tom) Thammavong

<https://github.com/ToMooGo/dimensionality-reduction-mlops>

October 2026

Abstract

This report documents an end-to-end project on dimensionality reduction. The methods are those of Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* (2nd ed.), Chapter 8. They are applied to the 70,000 MNIST digits. Principal Component Analysis keeps 95.02% of the training variance with 154 of 784 components. The decompressed test images keep 95.10% of the test variance. Incremental PCA fed from a memory-mapped file needs at most 24 MB of memory, against 569 MB for a full SVD with the data in RAM. Calling `fit()` on the memory map, as the book does, copies the whole array into memory. Kernel PCA is tuned over 20 kernel and γ settings, with two selection rules. The supervised rule reaches 96.4% held-out accuracy on the Swiss roll. The unsupervised rule (lowest pre-image error) chooses a different kernel, which reaches only 73.6%. t-SNE separates the digits far better than PCA, Kernel PCA, LLE, Isomap or MDS: 93.3% k -nearest-neighbour accuracy in 2D, against 41.4% for PCA and 92.8% for the same classifier on all 784 pixels. With PCA features, an RBF-kernel SVM trains 2.6 times and predicts 2.7 times faster, and its accuracy rises from 97.01% to 97.48%. An ablation shows that most of this gain comes from the kernel width that the default `gamma` implies on the new features, not from the compression itself. A Random Forest instead becomes slower and less accurate. Finally, the PCA reconstruction error flags unusual inputs at 3.34% false alarms. Everything runs as one Prefect pipeline with MLflow tracking, a model registry with quality gates, a FastAPI service and PostgreSQL monitoring, started by `docker compose up`.

Contents

1	Introduction	3
2	Data and evaluation protocol	3
3	Principal Component Analysis	3
3.1	Preserving the variance	4
3.2	The same axes minimise the reconstruction error	4
3.3	Computing the components with the SVD	4
3.4	Projection, decompression and the reconstruction error	5
3.5	Choosing the number of dimensions	5
3.6	“Less than 20% of its original size”: features or bytes?	6

4	PCA at scale	7
4.1	Solvers and their cost	7
4.2	Incremental PCA	7
4.3	Out-of-core training with a memory map	7
4.4	Results	8
5	Kernel PCA	9
5.1	PCA in a feature space	9
5.2	Pre-images	9
5.3	Selecting the kernel and γ	9
6	Manifold learning and 2D maps	11
6.1	Locally Linear Embedding	11
6.2	Isomap, MDS and t-SNE	11
6.3	How good is a 2D map?	11
6.4	Results	11
7	Compressed features for classification	14
8	Flagging unusual inputs with the reconstruction error	15
8.1	Score and threshold	15
8.2	Results and blind spots	16
9	From notebook to service	18
10	Limitations and next steps	19
A	Reproducing the results	20

1 Introduction

MNIST images have 784 pixels, but handwritten digits occupy a tiny part of that space. They are made of connected strokes on a white background, roughly centred. Chapter 8 of Géron’s book calls this the *manifold assumption*: most real high-dimensional data lie close to a much lower-dimensional manifold. Dimensionality reduction exploits it for three purposes: to compress data, to speed up the algorithms that come after it, and to visualise data in two dimensions.

The project asks six questions, labelled A–F throughout the repository:

- A. How many principal components keep 95% of MNIST’s variance, and how good are the decompressed images?
- B. Can PCA run on data that does not fit in memory, and do the faster solvers give the same answer?
- C. How should Kernel PCA be tuned when there is no obvious performance measure?
- D. Which 2D map of MNIST best separates the digits?
- E. Does training on compressed features make classifiers faster, and at what cost in accuracy?
- F. Can the reconstruction error of PCA flag inputs that do not look like digits?

Each question is answered with the book’s method, measured on held-out data, and reported with its limits. The models answering A, E and F are deployed behind a web service, together with the maps from D. Sections 3–8 derive the mathematics and present the results. Section 9 describes the system.

2 Data and evaluation protocol

MNIST is loaded as in the book with `fetch_openml("mnist_784")`: 70,000 images of 28×28 pixels with intensities from 0 (background) to 255 (ink). Following the book (exercise 9), the first 60,000 images form the training set and the last 10,000 the test set. When OpenML cannot be reached, the original MNIST files are downloaded from a mirror instead. They are accepted only if their MD5 checksums match the official ones, and they give the same matrix in the same order.

All randomness is seeded. Every model is fitted on training data only, and every quality figure is measured on the test set unless stated otherwise. Timings are wall-clock seconds on a 2-CPU machine (Python 3.13.16, scikit-learn 1.9.1, NumPy 2.5.3). They are medians of three runs; fits slower than 30 s (Section 4) or 60 s (Section 7) are timed once. They compare methods on the same machine and are not absolute benchmarks.

A test-set number is a point estimate. Following the book (Chapter 2, p. 83), its precision is given by a 95% confidence interval computed with `scipy.stats.t.interval` from the per-image values. For an accuracy the per-image value is 1 for a correct prediction and 0 otherwise. For the difference between two models evaluated on the same images it is the per-image difference of those values, which gives a *paired* interval.

3 Principal Component Analysis

Let $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)} \in \mathbb{R}^n$ be the training images ($m = 60,000$, $n = 784$). Let

$$\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)}, \quad \mathbf{S} = \frac{1}{m-1} \sum_{i=1}^m (\mathbf{x}^{(i)} - \boldsymbol{\mu})(\mathbf{x}^{(i)} - \boldsymbol{\mu})^\top \quad (1)$$

be their mean and sample covariance matrix. $\mathbf{X}_c$ denotes the $m \times n$ matrix of centred images, so that $\mathbf{S} = \mathbf{X}_c^\top \mathbf{X}_c / (m - 1)$.

3.1 Preserving the variance

PCA looks for the unit vector $\mathbf{c}_1$ along which the projected data vary most. The projection of a centred image on $\mathbf{c}$ is $\mathbf{c}^\top (\mathbf{x}^{(i)} - \boldsymbol{\mu})$, and its sample variance is

$$\frac{1}{m-1} \sum_{i=1}^m (\mathbf{c}^\top (\mathbf{x}^{(i)} - \boldsymbol{\mu}))^2 = \frac{1}{m-1} \sum_{i=1}^m \mathbf{c}^\top (\mathbf{x}^{(i)} - \boldsymbol{\mu}) (\mathbf{x}^{(i)} - \boldsymbol{\mu})^\top \mathbf{c} = \mathbf{c}^\top \mathbf{S} \mathbf{c}. \quad (2)$$

We maximise $\mathbf{c}^\top \mathbf{S} \mathbf{c}$ subject to $\mathbf{c}^\top \mathbf{c} = 1$ with a Lagrange multiplier λ :

$$\mathcal{L}(\mathbf{c}, \lambda) = \mathbf{c}^\top \mathbf{S} \mathbf{c} - \lambda (\mathbf{c}^\top \mathbf{c} - 1), \quad (3)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}} = 2\mathbf{S} \mathbf{c} - 2\lambda \mathbf{c} = \mathbf{0} \implies \mathbf{S} \mathbf{c} = \lambda \mathbf{c}. \quad (4)$$

So $\mathbf{c}$ must be an eigenvector of $\mathbf{S}$. Its variance is then $\mathbf{c}^\top \mathbf{S} \mathbf{c} = \lambda \mathbf{c}^\top \mathbf{c} = \lambda$, so the maximum is reached by the eigenvector with the largest eigenvalue λ_1 . The second component maximises the same quantity under the extra constraint $\mathbf{c}_2^\top \mathbf{c}_1 = 0$. Adding a multiplier ϕ for it gives $2\mathbf{S} \mathbf{c}_2 - 2\lambda \mathbf{c}_2 - \phi \mathbf{c}_1 = \mathbf{0}$. Multiplying on the left by $\mathbf{c}_1^\top$ and using $\mathbf{c}_1^\top \mathbf{S} = \lambda_1 \mathbf{c}_1^\top$ and $\mathbf{c}_1^\top \mathbf{c}_2 = 0$ gives $\phi = 0$. Hence $\mathbf{c}_2$ is the eigenvector with the second largest eigenvalue, and so on. Because $\mathbf{S}$ is symmetric, its eigenvectors are orthonormal and its eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ are real.

3.2 The same axes minimise the reconstruction error

The book gives a second justification: the chosen axes also minimise the mean squared distance between the data and their projection. Let $\mathbf{W}_d = [\mathbf{c}_1, \dots, \mathbf{c}_d]$ have orthonormal columns ($\mathbf{W}_d^\top \mathbf{W}_d = \mathbf{I}_d$), and write $\mathbf{u} = \mathbf{x} - \boldsymbol{\mu}$. The projection onto the subspace is $\mathbf{W}_d \mathbf{W}_d^\top \mathbf{u}$, and the squared residual is

$$\begin{aligned} \|\mathbf{u} - \mathbf{W}_d \mathbf{W}_d^\top \mathbf{u}\|^2 &= \mathbf{u}^\top \mathbf{u} - 2\mathbf{u}^\top \mathbf{W}_d \mathbf{W}_d^\top \mathbf{u} + \mathbf{u}^\top \mathbf{W}_d \underbrace{\mathbf{W}_d^\top \mathbf{W}_d}_{\mathbf{I}_d} \mathbf{W}_d^\top \mathbf{u} \\ &= \|\mathbf{u}\|^2 - \|\mathbf{W}_d^\top \mathbf{u}\|^2. \end{aligned} \quad (5)$$

Summing over the training set and dividing by $m - 1$,

$$\frac{1}{m-1} \sum_{i=1}^m \|\mathbf{u}^{(i)} - \mathbf{W}_d \mathbf{W}_d^\top \mathbf{u}^{(i)}\|^2 = \text{tr}(\mathbf{S}) - \text{tr}(\mathbf{W}_d^\top \mathbf{S} \mathbf{W}_d). \quad (6)$$

$\text{tr}(\mathbf{S})$ does not depend on $\mathbf{W}_d$. Minimising the residual is therefore the same as maximising the projected variance $\text{tr}(\mathbf{W}_d^\top \mathbf{S} \mathbf{W}_d)$, and the first d eigenvectors do both.

3.3 Computing the components with the SVD

The book computes PCA with the singular value decomposition $\mathbf{X}_c = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^\top$ (Equation 8-1). Then

$$\mathbf{S} = \frac{\mathbf{X}_c^\top \mathbf{X}_c}{m-1} = \frac{\mathbf{V} \boldsymbol{\Sigma} \mathbf{U}^\top \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^\top}{m-1} = \mathbf{V} \frac{\boldsymbol{\Sigma}^2}{m-1} \mathbf{V}^\top, \quad (7)$$

because $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$. This is an eigendecomposition of $\mathbf{S}$. The columns of $\mathbf{V}$ are the principal components, and the eigenvalues are $\lambda_j = \sigma_j^2 / (m - 1)$. The *explained variance ratio* of component j is $\lambda_j / \sum_k \lambda_k$. The first notebook checks that the book's NumPy code and scikit-learn find the same first two components up to sign; the cosine between them is 1 to six decimals. The first two components explain 9.7% and 7.1% of the variance.

3.4 Projection, decompression and the reconstruction error

Compression projects a centred image onto the first d components (Equation 8-2). Decompression maps the code back (Equation 8-3):

$$\mathbf{z} = \mathbf{W}_d^\top (\mathbf{x} - \boldsymbol{\mu}) \in \mathbb{R}^d, \quad \hat{\mathbf{x}} = \mathbf{W}_d \mathbf{z} + \boldsymbol{\mu}. \quad (8)$$

The book defines the reconstruction error as the mean squared distance between $\mathbf{x}$ and $\hat{\mathbf{x}}$. Its value on the training set follows from the derivation above. Write $\mathbf{x}^{(i)} - \boldsymbol{\mu} = \sum_{j=1}^n a_{ij} \mathbf{c}_j$ in the eigenbasis. Then $\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)} = \sum_{j>d} a_{ij} \mathbf{c}_j$, and by orthonormality $\|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|^2 = \sum_{j>d} a_{ij}^2$. Averaging over images and pixels,

$$\text{MSE}_{\text{train}} = \frac{1}{mn} \sum_{i=1}^m \sum_{j>d} a_{ij}^2 = \frac{1}{mn} \sum_{j>d} (m-1) \lambda_j = \frac{m-1}{m} \cdot \frac{1}{n} \sum_{j>d} \lambda_j, \quad (9)$$

where we used $\sum_i a_{ij}^2 = (m-1) \lambda_j$, since a_{ij} is the centred coordinate along $\mathbf{c}_j$. *The training reconstruction error is the variance of the dropped components per pixel.* The pipeline checks this identity on every run. At $d = 154$ both sides equal 217.79.

3.5 Choosing the number of dimensions

The book’s rule picks the smallest d whose cumulative explained variance reaches 95%. On MNIST this gives $d = 154$, 19.6% of the 784 features, which is the book’s “just over 150”. `PCA(n_components=0.95)` returns the same d . Figure 1 shows the curve, and Table 1 gives several compression levels. The test set keeps slightly *more* of its variance than the training set (95.10% vs 95.02% at 95%), so the compression generalises. The typical error of a test pixel is 14.7 grey levels out of 255, and Figure 2 shows that the digits remain clearly legible.

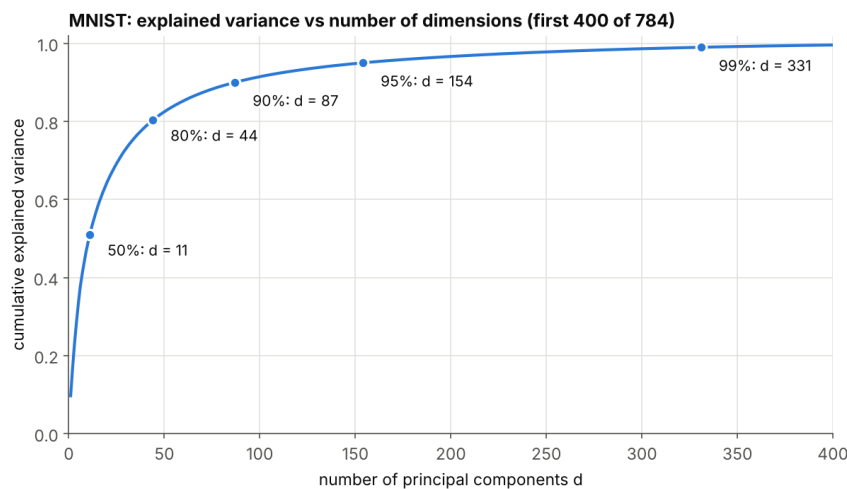


Figure 1: Cumulative explained variance of MNIST’s principal components, with the number of components needed for 50%, 80%, 90%, 95% and 99% of the variance.

Table 1: Compression levels. “Test variance” is the share of the test set’s variance that survives compression followed by decompression; RMSE is in grey levels (0–255).

target	d	features	train var.	test var.	test RMSE
50%	11	1.4%	50.92%	51.55%	46.1
80%	44	5.6%	80.33%	80.87%	29.0
90%	87	11.1%	90.01%	90.27%	20.7
95%	154	19.6%	95.02%	95.10%	14.7
99%	331	42.2%	99.00%	99.02%	6.6

Test digits compressed to d components and decompressed (inverse_transform)

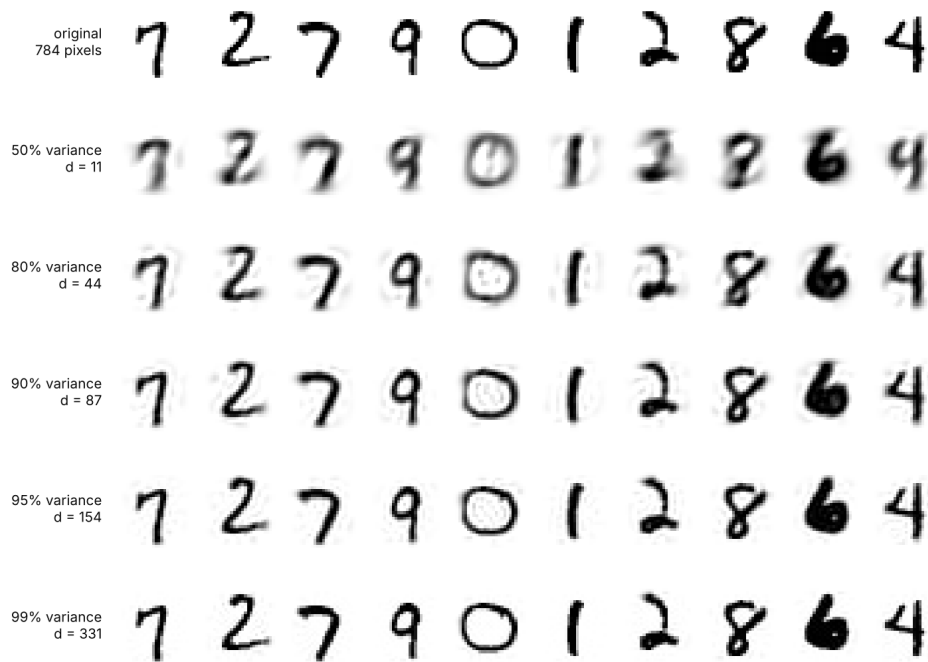


Figure 2: Ten test digits, compressed to d components and decompressed.

3.6 “Less than 20% of its original size”: features or bytes?

The book’s figure compares numbers of features: $154/784 = 19.6\%$. MNIST, however, is stored with one byte per pixel, while PCA codes are floating-point numbers. Table 2 shows that float32 codes take 78.6% of the bytes of the original uint8 file, and float16 codes 39.3%. The saving in memory depends on how both sides are stored.

Table 2: Storage per image for the raw pixels and for the 154-value PCA codes.

representation	bytes/image	vs float32 pixels	vs uint8 pixels
raw pixels, uint8 (as shipped)	784	25.0%	100.0%
raw pixels, float32	3,136	100.0%	400.0%
PCA codes, float32 (154 values)	616	19.6%	78.6%
PCA codes, float16 (154 values)	308	9.8%	39.3%

4 PCA at scale

4.1 Solvers and their cost

The book compares the full SVD, with cost $O(mn^2) + O(n^3)$, and Randomized PCA, which approximates the first d components in $O(md^2) + O(d^3)$. Both need the whole training matrix in memory. In practice each pass of the randomized algorithm also multiplies the $m \times n$ data by an $n \times (d + p)$ block, which costs $O(mnd)$. With $d/n = 154/784 \approx 0.2$ the saving over the exact methods is therefore small on MNIST. Since version 1.5, scikit-learn's `svd_solver="auto"` also considers a third route for tall data ($m \gg n$). It forms the $n \times n$ covariance matrix in $O(mn^2)$ and eigendecomposes it in $O(n^3)$, which is exactly the eigenproblem of Section 3. On MNIST, "auto" picked `covariance_eigh`.

4.2 Incremental PCA

Incremental PCA processes mini-batches one at a time, so the full matrix never has to be in memory. The idea can be seen in the statistics it must keep up to date. Suppose batch A (n_A images, mean μ_A , scatter matrix $\mathbf{M}_A = \sum_{i \in A} (\mathbf{x}^{(i)} - \mu_A)(\mathbf{x}^{(i)} - \mu_A)^\top$) has been processed, and batch B arrives. The combined mean is

$$\mu_{AB} = \frac{n_A \mu_A + n_B \mu_B}{n_A + n_B}. \quad (10)$$

For the combined scatter, write $\mathbf{x} - \mu_{AB} = (\mathbf{x} - \mu_A) + (\mu_A - \mu_{AB})$ for $\mathbf{x} \in A$. The cross terms vanish because $\sum_{i \in A} (\mathbf{x}^{(i)} - \mu_A) = \mathbf{0}$, so

$$\sum_{i \in A} (\mathbf{x}^{(i)} - \mu_{AB})(\mathbf{x}^{(i)} - \mu_{AB})^\top = \mathbf{M}_A + n_A (\mu_A - \mu_{AB})(\mu_A - \mu_{AB})^\top, \quad (11)$$

and similarly for B . Using $\mu_A - \mu_{AB} = \frac{n_B}{n_A + n_B}(\mu_A - \mu_B)$ and $\mu_B - \mu_{AB} = -\frac{n_A}{n_A + n_B}(\mu_A - \mu_B)$, the two correction terms add up to

$$\mathbf{M}_{AB} = \mathbf{M}_A + \mathbf{M}_B + \frac{n_A n_B}{n_A + n_B} (\mu_A - \mu_B)(\mu_A - \mu_B)^\top. \quad (12)$$

The combined statistics therefore depend only on the summaries of each batch, not on the images themselves. scikit-learn's `IncrementalPCA` (the algorithm of Ross et al. cited by the book) keeps a rank- d SVD summary instead of the full $n \times n$ matrix and applies the same mean correction. This is why its result differs slightly from the exact components.

4.3 Out-of-core training with a memory map

The book stores the data in a binary file, opens it with `np.memmap` and passes the memory map `X_mm` to the `fit()` method of an `IncrementalPCA` with 154 components. Measuring the peak memory of that call gives 212 MB. The measure is the peak of the heap allocations that NumPy reports to Python's `tracemalloc`. It does not count pages of the memory-mapped file, which the operating system can drop at any time, or the BLAS library's own small internal buffers. That is the whole 188 MB training matrix. The reason is that `fit()` first validates its input with `copy=True`, which copies the memory map into RAM. Calling `partial_fit` on one slice of the memory map at a time keeps the peak at 24 MB and gives identical components. This is 24 times less memory than a full SVD with the data in RAM (569 MB).

4.4 Results

Table 3 and Figure 3 summarise the six variants. All of them find essentially the same subspace: the test reconstruction error of Randomized PCA is 0.52% above the exact SVD, and that of Incremental PCA 0.81%. Randomized PCA is not dramatically faster here (2.69 s vs 3.17 s), because with only $n = 784$ features the full SVD is already cheap. The book's complexity argument pays off when n is large. The covariance route chosen by "auto" is the fastest (0.33 s) and needs little memory beyond the data. Incremental PCA is the slowest solver (19.1 s), and the price of the out-of-core version is time, not accuracy.

Table 3: 154 components of the 60,000 training images. RAM = peak allocations while fitting, plus the training matrix when it is held in memory. Δ test MSE is relative to the exact SVD.

solver	data in	fit (s)	RAM (MB)	variance	Δ test MSE
PCA, full SVD	RAM	3.17	569	0.9502	+0.00%
PCA, solver='auto'	RAM	0.33	215	0.9502	-0.00%
Randomized PCA	RAM	2.69	495	0.9498	+0.52%
Incremental PCA, partial_fit	RAM	19.08	212	0.9495	+0.81%
Incremental PCA, memmap + fit()	disk	19.24	212	0.9495	+0.81%
Incremental PCA, memmap + partial_fit	disk	18.75	24	0.9495	+0.81%

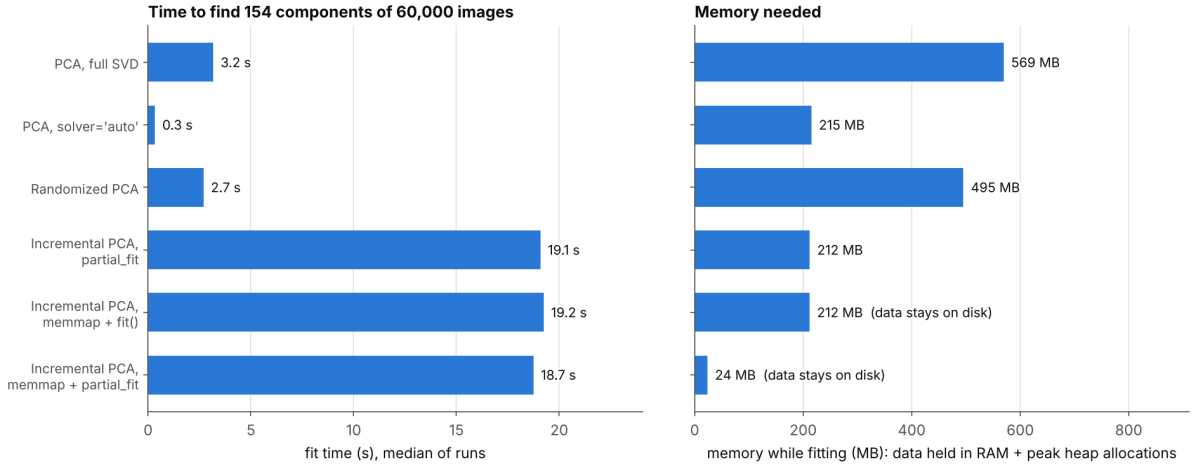


Figure 3: Fit time and memory of the six PCA variants.

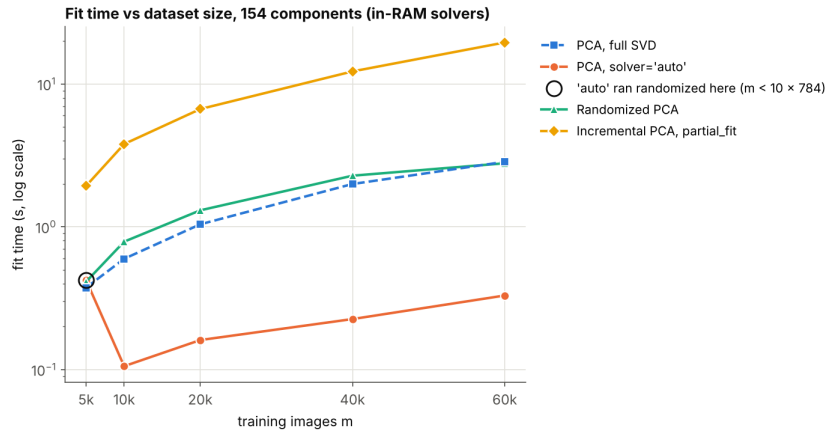


Figure 4: Fit time against the number of training images (log scale).

5 Kernel PCA

5.1 PCA in a feature space

Kernel PCA applies PCA after an implicit map $\phi : \mathbb{R}^n \rightarrow \mathcal{F}$ into a high-dimensional feature space. Only inner products $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$ are computed: this is the kernel trick of Chapter 5. The kernels used here are the RBF kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ and the sigmoid kernel $k(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^\top \mathbf{x}' + r)$.

Assume first that the mapped points are centred, $\sum_i \phi(\mathbf{x}^{(i)}) = \mathbf{0}$. The covariance in feature space is $\mathbf{C} = \frac{1}{m} \sum_i \phi(\mathbf{x}^{(i)}) \phi(\mathbf{x}^{(i)})^\top$. An eigenvector $\mathbf{v}$ with eigenvalue $\lambda > 0$ satisfies

$$\mathbf{v} = \frac{1}{\lambda} \mathbf{C} \mathbf{v} = \sum_{i=1}^m \underbrace{\frac{\phi(\mathbf{x}^{(i)})^\top \mathbf{v}}{m\lambda}}_{\alpha_i} \phi(\mathbf{x}^{(i)}), \quad (13)$$

so $\mathbf{v}$ lies in the span of the mapped training points. Substitute $\mathbf{v} = \sum_j \alpha_j \phi(\mathbf{x}^{(j)})$ into $\mathbf{C} \mathbf{v} = \lambda \mathbf{v}$ and take the inner product with $\phi(\mathbf{x}^{(l)})$ on both sides. With the kernel matrix $K_{ij} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ this gives

$$\frac{1}{m} \sum_i K_{li} \sum_j K_{ij} \alpha_j = \lambda \sum_j K_{lj} \alpha_j \iff \mathbf{K}^2 \boldsymbol{\alpha} = m\lambda \mathbf{K} \boldsymbol{\alpha}, \quad (14)$$

which is solved by the eigenproblem $\mathbf{K} \boldsymbol{\alpha} = m\lambda \boldsymbol{\alpha}$. The normalisation $\mathbf{v}^\top \mathbf{v} = 1$ becomes $\boldsymbol{\alpha}^\top \mathbf{K} \boldsymbol{\alpha} = m\lambda \boldsymbol{\alpha}^\top \boldsymbol{\alpha} = 1$. The coordinate of any point along $\mathbf{v}$ needs kernel values only:

$$z(\mathbf{x}) = \mathbf{v}^\top \phi(\mathbf{x}) = \sum_{i=1}^m \alpha_i k(\mathbf{x}^{(i)}, \mathbf{x}). \quad (15)$$

If the mapped points are not centred, ϕ is replaced by $\tilde{\phi}(\mathbf{x}) = \phi(\mathbf{x}) - \frac{1}{m} \sum_j \phi(\mathbf{x}^{(j)})$. Expanding $\tilde{\phi}(\mathbf{x}^{(i)})^\top \tilde{\phi}(\mathbf{x}^{(j)})$ gives the centred kernel matrix

$$\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_m \mathbf{K} - \mathbf{K} \mathbf{1}_m + \mathbf{1}_m \mathbf{K} \mathbf{1}_m, \quad (\mathbf{1}_m)_{ij} = \frac{1}{m}. \quad (16)$$

A linear kernel $k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$ gives back ordinary PCA (Figure 5, second panel).

5.2 Pre-images

A point in the 2D kernel-PCA space maps back to a point of the feature space, which in general is not the image of any input. The book therefore uses an approximate *pre-image*. A regression model learns the map from the 2D coordinates back to the inputs; scikit-learn uses kernel ridge regression when `fit_inverse_transform=True`. The *pre-image error* is the mean squared distance between inputs and their pre-images. With the book's setting (RBF kernel, $\gamma = 0.0433$, all 1,000 points of the Swiss roll) the pipeline obtains 32.7863. The book prints 32.786, so this reproduction matches it exactly.

5.3 Selecting the kernel and γ

The book proposes two rules. The *supervised* rule chooses the setting that maximises the accuracy of a downstream task: a `KernelPCA` $\rightarrow$ `LogisticRegression` pipeline is tuned by `GridSearchCV` over two kernels and ten values of γ between 0.03 and 0.05, 20 settings in total. The *unsupervised* rule chooses the setting with the lowest pre-image error. The task is the book's: separate the two halves of a 1,000-point Swiss roll, $t > 6.9$ along the roll. The book fits on all points. Here 25% of the roll is held out: both rules choose using the other 75% only, the unsupervised one by the 3-fold cross-validated pre-image error, and both choices are then judged on the held-out points.

The supervised rule chooses rbf with $\gamma = 0.0411$ (book: 0.0433). Its cross-validated accuracy is $96.1\% \pm 1.0\%$, and its held-out accuracy 96.4%. 7 of the 20 settings lie within one standard deviation of the best, so the exact γ is not significant: any RBF γ in the upper half of the grid does as well. Logistic Regression on the raw 3D coordinates reaches 64.8%, and after linear PCA to 2D 75.2%.

The unsupervised rule chooses sigmoid with $\gamma = 0.0478$, whose cross-validated pre-image error is 21.76. The same pipeline with this setting reaches only 73.6% on the held-out points (Figure 6), close to linear PCA. The reason is visible in the kernel itself. For small γ , $\tanh(\gamma \mathbf{x}^\top \mathbf{x}' + 1) \approx \tanh(1) + (1 - \tanh^2 1) \gamma \mathbf{x}^\top \mathbf{x}'$. This is an affine function of the linear kernel, so the sigmoid kernel behaves almost like linear PCA. It rebuilds the inputs well, but it does not unroll the roll. *The two rules answer different questions.* A small pre-image error says the 2D code keeps enough information to rebuild the input. It does not say that the 2D layout makes the task easy, so the purpose of the reduction should decide which rule to use. (The book's in-sample error of 32.786, reproduced above, is measured on the fitting points and is not comparable with the cross-validated values.)

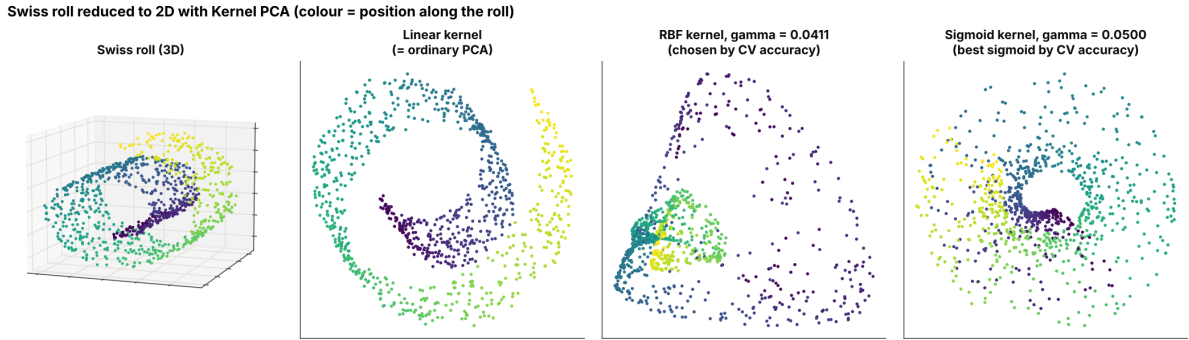


Figure 5: The Swiss roll and three Kernel PCA projections (colour = position t along the roll).

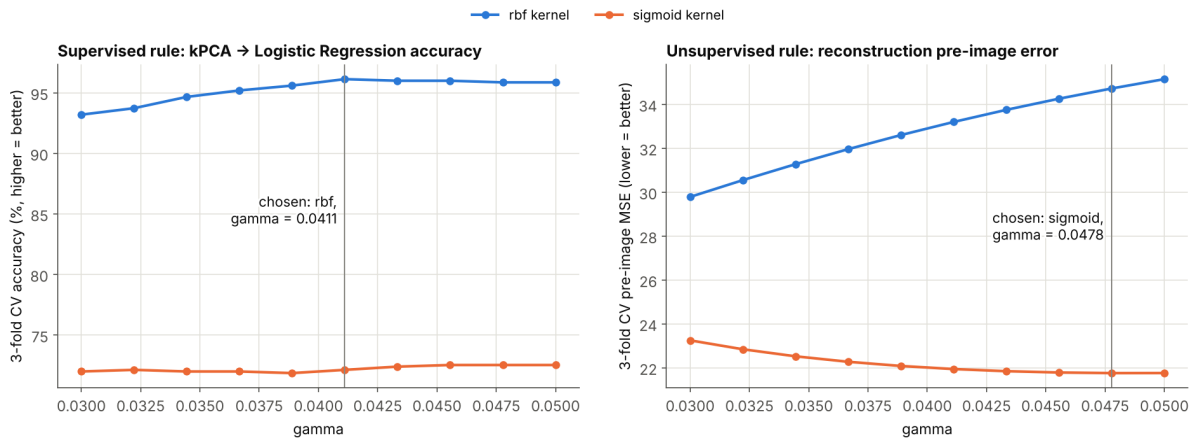


Figure 6: Left: supervised selection by cross-validated accuracy. Right: unsupervised selection by held-out pre-image error. Each rule picks a different kernel.

6 Manifold learning and 2D maps

6.1 Locally Linear Embedding

LLE first reconstructs every point from its k nearest neighbours (Equation 8-4) and then finds low-dimensional points that keep those reconstruction weights (Equation 8-5).

Step 1. For one point $\mathbf{x}$ with neighbours $\boldsymbol{\eta}_1, \dots, \boldsymbol{\eta}_k$ and weights summing to one,

$$\left\| \mathbf{x} - \sum_j w_j \boldsymbol{\eta}_j \right\|^2 = \left\| \sum_j w_j (\mathbf{x} - \boldsymbol{\eta}_j) \right\|^2 = \sum_{j,l} w_j w_l G_{jl}, \quad G_{jl} = (\mathbf{x} - \boldsymbol{\eta}_j)^\top (\mathbf{x} - \boldsymbol{\eta}_l), \quad (17)$$

where we used $\sum_j w_j = 1$ to move $\mathbf{x}$ inside the sum. Minimising $\mathbf{w}^\top \mathbf{G} \mathbf{w}$ subject to $\mathbf{1}^\top \mathbf{w} = 1$ with a multiplier ν gives $2\mathbf{G}\mathbf{w} = \nu \mathbf{1}$. Hence $\mathbf{w} \propto \mathbf{G}^{-1} \mathbf{1}$, and the constraint fixes the scale:

$$\mathbf{w} = \frac{\mathbf{G}^{-1} \mathbf{1}}{\mathbf{1}^\top \mathbf{G}^{-1} \mathbf{1}}. \quad (18)$$

($\mathbf{G}$ is regularised when k exceeds the dimension.)

Step 2. With the weight matrix $\mathbf{W}$ fixed, write the cost of Equation 8-5 as $\sum_i \|\mathbf{z}^{(i)} - \sum_j w_{ij} \mathbf{z}^{(j)}\|^2 = \|(\mathbf{I} - \mathbf{W})\mathbf{Z}\|_F^2 = \text{tr}(\mathbf{Z}^\top \mathbf{M} \mathbf{Z})$ with $\mathbf{M} = (\mathbf{I} - \mathbf{W})^\top (\mathbf{I} - \mathbf{W})$. To exclude the trivial solution $\mathbf{Z} = \mathbf{0}$, impose $\frac{1}{m} \mathbf{Z}^\top \mathbf{Z} = \mathbf{I}$. The minimiser is then given by the eigenvectors of $\mathbf{M}$ with the smallest eigenvalues. The constant vector, with eigenvalue 0 because each row of $\mathbf{W}$ sums to one, is discarded.

6.2 Isomap, MDS and t-SNE

The book describes three more methods. Multidimensional Scaling (MDS) keeps the pairwise distances. Isomap keeps *geodesic* distances along a nearest-neighbour graph. t-SNE keeps similar instances close and dissimilar ones apart, and is mostly used for visualisation. t-SNE and MDS have no **transform** method, so they can only place the points they were fitted on. This matters for the deployed service (Section 9).

6.3 How good is a 2D map?

Exercise 7 of the book answers this question: a reduction can be judged by the performance of the algorithm that uses it. On the Swiss roll, a standardised Logistic Regression is trained on the 2D coordinates to tell the two halves of the roll apart. On MNIST, a 5-nearest-neighbour classifier is trained on the 2D coordinates. In both cases the score is the cross-validated accuracy. A high score means that points of the same class form their own regions.

6.4 Results

On the Swiss roll (Table 4, Figure 7), Isomap (99.2%), t-SNE (98.7%) and LLE (98.2%) separate the two halves almost perfectly. PCA (71.6%) and MDS (75.1%) cannot, because they preserve straight-line distances across the layers of the roll. LLE's best axis follows the position along the roll almost exactly (correlation 0.997), but the spacing is uneven, as the book notes.

Table 4: Manifold methods on the Swiss roll.

method	time (s)	downstream accuracy	$\max_j \text{corr}(z_j, t) $
PCA	<0.01	71.6%	0.219
Kernel PCA (rbf)	0.09	96.4%	0.461
LLE	0.07	98.2%	0.997
Isomap	0.15	99.2%	0.992
MDS	1.90	75.1%	0.179
t-SNE	2.32	98.7%	0.904

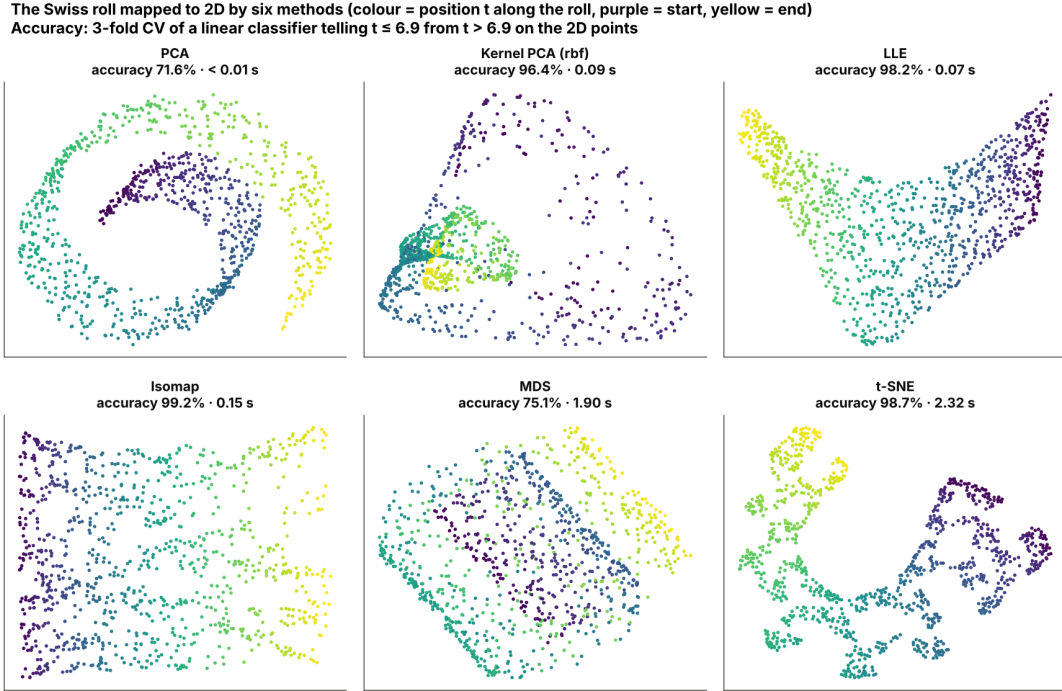


Figure 7: The Swiss roll mapped to 2D by six methods.

On MNIST (5,000 training digits; Table 5 and Figures 8 and 9), t-SNE is in a class of its own, with 93.3% kNN accuracy in 2D. The next best is LLE with 68.2%. PCA reaches 41.4% and MDS 45.3%, and MDS takes 191 s. The nonlinear methods run after a PCA that keeps 95% of the variance, as in the book’s exercise 8. The control run of t-SNE on the raw pixels shows that this chaining costs nothing: 92.9% in 16.7 s, against 93.3% in 16.5 s. For reference, the same classifier on all 784 pixels reaches $92.8\% \pm 0.7\%$ (standard deviation over the five folds), so in two numbers per image t-SNE separates the digits as well as the full pixel vectors do. This comparison flatters t-SNE a little. Because t-SNE and MDS cannot place new points, every map is fitted on all 5,000 points before the cross-validation, so the test folds have already shaped the layout. The score therefore describes how well a map separates the digits; it is not a held-out classification accuracy.

Table 5: 2D maps of 5,000 MNIST training digits.

method	time (s)	kNN accuracy ($\pm$ std over folds)
PCA	0.2	41.4% $\pm$ 1.4
Kernel PCA	1.3	40.7% $\pm$ 1.9
LLE	5.3	68.2% $\pm$ 1.2
Isomap	5.9	50.4% $\pm$ 1.6
MDS	190.5	45.3% $\pm$ 0.8
t-SNE	16.5	93.3% $\pm$ 0.9
t-SNE (raw pixels)	16.7	92.9% $\pm$ 0.8
<i>no reduction: all 784 pixels</i>	–	92.8% $\pm$ 0.7

t-SNE of 5,000 MNIST training digits (PCA to 95% variance first)

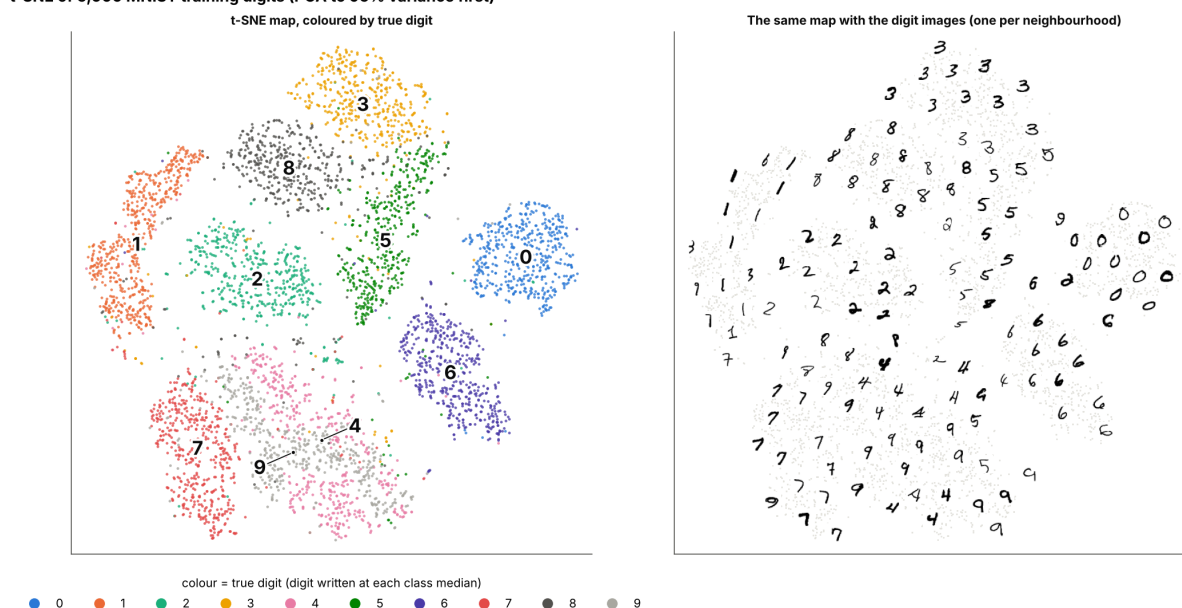


Figure 8: t-SNE map of MNIST. Right: one digit image wherever no other image is already close (the book's suggestion in exercise 10).

5,000 MNIST digits mapped to 2D: higher kNN accuracy = digits better separated

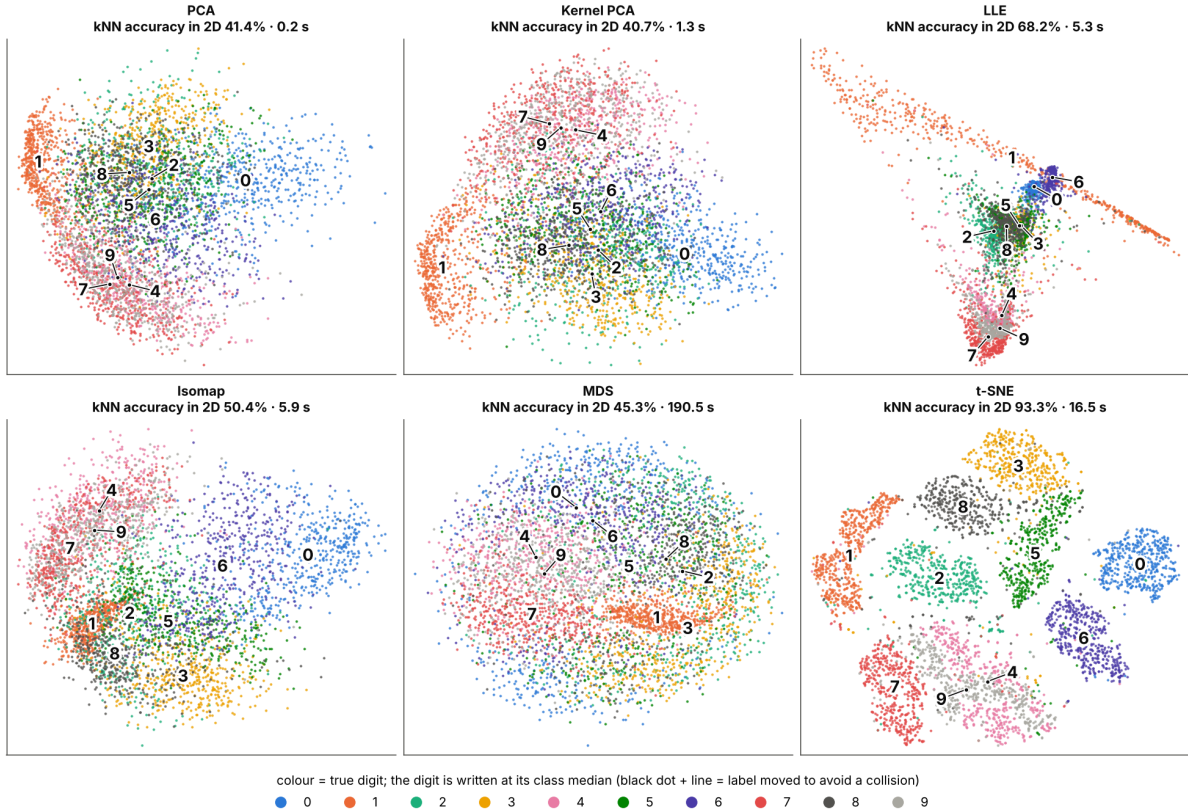


Figure 9: The same digits mapped by six methods.

7 Compressed features for classification

Exercise 9 of the book trains a Random Forest on the raw pixels and on the PCA features, and compares training time and accuracy. The same comparison is run for Softmax Regression and for an RBF-kernel SVM, the classifier the book says PCA can speed up “tremendously”. The SVM is trained on 20,000 random training images, because kernel SVMs scale roughly quadratically with the number of samples. The PCA is unsupervised, so it is fitted on all 60,000 training images (154 components, the same as in Section 3), and its fit time is included in the training time. Both SVMs use scikit-learn’s defaults, $C = 1$ and `gamma="scale"`; neither is tuned. Table 6 and Figure 10 give the results.

Table 6: Classifiers on raw pixels and on PCA features (95% of the variance), evaluated on the 10,000 test images.

classifier	features	train	fit (s)	predict (s)	accuracy (95% CI)	MB
Softmax Regression	784 pixels	60,000	34.1	0.01	92.64 (92.13–93.15)	0.0
Softmax Regression	PCA (154)	60,000	4.1	0.04	92.32 (91.80–92.84)	0.5
Random Forest	784 pixels	60,000	29.8	0.27	97.05 (96.72–97.38)	146.1
Random Forest	PCA (154)	60,000	72.1	0.24	94.95 (94.52–95.38)	212.8
SVM	784 pixels	20,000	22.9	26.21	97.01 (96.68–97.34)	39.3
SVM	PCA (154)	20,000	8.7	9.88	97.48 (97.17–97.79)	8.6
SVM	PCA, raw γ (154)	20,000	7.6	9.07	97.13 (96.80–97.46)	8.2

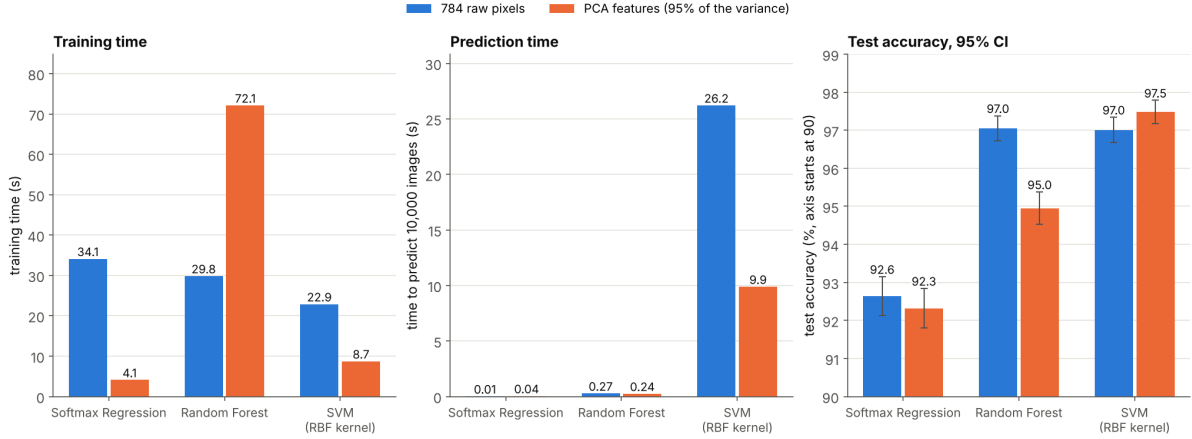


Figure 10: Training time, prediction time and accuracy with and without PCA.

- **SVM.** PCA makes training 2.6 times and prediction 2.7 times faster. Evaluating an RBF kernel costs $O(n)$ per pair of points, so cutting n from 784 to 154 cuts the cost of every kernel evaluation. The stored model also shrinks, from 39.3 MB to 8.6 MB, because each support vector is now stored as 154 numbers instead of 784. Accuracy changes from 97.01% to 97.48%, a difference of +0.47 pp with a paired 95% interval of +0.31 to +0.63 pp.

The accuracy change has a second cause besides the compression. `gamma="scale"` sets $\gamma = 1/(n \text{Var}(\mathbf{X}))$, which gives a different kernel width on the PCA features than on the pixels. An ablation therefore runs the PCA SVM with the raw model's γ . Since PCA nearly preserves the distances between images, the two models then use almost the same kernel. This variant scores 97.13%, +0.12 pp against raw pixels (95% interval +0.01 to +0.23 pp). Of the 0.47 pp gain, only 0.12 pp therefore remains when the kernel width is held fixed, and the lower end of its interval is close to zero. Most of the accuracy gain comes from the different bandwidth that `gamma="scale"` chooses on the PCA features. The speed-up, in contrast, is a property of PCA itself. Tuning γ for each representation would be the fair next comparison.

- **Softmax Regression.** Training is 8.3 times faster, and accuracy changes by -0.32 pp (95% interval -0.61 to -0.03 pp).
- **Random Forest.** Training is *slower* with PCA (72.1 s against 29.8 s), and accuracy drops from 97.05% to 94.95% (-2.10 pp, 95% interval -2.46 to -1.74 pp), as the book's exercise warns. The stored forest also grows from 146 MB to 213 MB, so its trees have many more nodes. A plausible explanation, not isolated here, is that each principal component mixes all pixels: a single split on it separates the classes less cleanly than a split on one well-placed pixel, so the trees need more splits.

Whether compression helps therefore depends on the model that comes after it. This is why the deployed pair is the SVM, and why the API serves both models side by side.

8 Flagging unusual inputs with the reconstruction error

8.1 Score and threshold

The detector combines two ideas from the book. The first is the PCA reconstruction error of Chapter 8: an input is scored by

$$e(\mathbf{x}) = \frac{1}{n} \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \frac{1}{n} \|(\mathbf{I} - \mathbf{W}_d \mathbf{W}_d^\top)(\mathbf{x} - \boldsymbol{\mu})\|^2. \quad (19)$$

Digits resemble the training data and are rebuilt well; inputs unlike them are not. The second is the percentile rule of Chapter 9 (p. 266): flag the 4% worst-scoring instances. The PCA ($d = 154$) is fitted on 50,000 training images. The threshold τ is the 96th percentile of e on the other 10,000 training images, $\tau = 427$. If new clean digits come from the same distribution as these held-out images, the probability that one exceeds τ is 4%, up to a correction of order $1/10,000$. A threshold set on the fitting images ($\tau = 420$) would be optimistic in general. With 50,000 images, however, PCA hardly overfits and both thresholds give similar rates: 3.34% (95% interval 2.99–3.69%) and 3.66% false alarms on the test digits. Both are below 4% because MNIST’s test digits are rebuilt slightly better than its training digits (Table 1).

8.2 Results and blind spots

Each test image is corrupted in eight ways (Table 7, Figures 11 and 12). Random noise, inverted, shuffled and Gaussian-noise images are all flagged, and 90.2% of the shifted ones. Upside-down (12.7%) and mirrored (17.2%) digits are mostly missed: they are still made of digit-like strokes, which the components can rebuild. Dimmed digits are never flagged (0.0%). The reason follows from the score. For $\tilde{\mathbf{x}} = \alpha\mathbf{x}$ with $0 < \alpha < 1$,

$$(\mathbf{I} - \mathbf{W}_d \mathbf{W}_d^\top)(\alpha\mathbf{x} - \boldsymbol{\mu}) = \alpha(\mathbf{I} - \mathbf{W}_d \mathbf{W}_d^\top)(\mathbf{x} - \boldsymbol{\mu}) - (1 - \alpha)(\mathbf{I} - \mathbf{W}_d \mathbf{W}_d^\top)\boldsymbol{\mu}. \quad (20)$$

When the mean image is itself well rebuilt, the second term is small and $e(\alpha\mathbf{x}) \approx \alpha^2 e(\mathbf{x})$. For $\alpha = 0.3$ this predicts a 91% *smaller* error. The measured median falls from 198 to 18, in agreement. A low-intensity input looks *more* normal to this score, so a second, lower threshold or an intensity check would be needed to catch it.

Over the eight corruptions the mean detection rate is 65.0%, and the ROC-AUC between clean and corrupted inputs is 0.791. These two averages depend on which corruptions are included, so the per-type rates are the informative numbers. All of them are synthetic; no real out-of-distribution images (letters, other datasets) were tested. The 4% target is the book’s rule of thumb, not a choice derived from the cost of a false alarm. A detector with fewer components separates them slightly better: 80% of the variance ($d = 44$) gives an AUC of 0.819 (Table 8). The deployed SVM is wrong on 6.0% of the 334 clean test digits that are flagged, and on 2.4% of the rest. The flag therefore marks harder inputs, but it catches only 20 of the classifier’s 252 errors. It is a guard against inputs that are not digits, not an estimate of the classifier’s confidence.

Table 7: Flag rate at the deployed threshold.

input (10,000 test images)	flagged	median error
clean test digits	3.3%	198
noise	100.0%	10,908
inverted	100.0%	26,459
shuffled	100.0%	5,297
rotated	12.7%	246
mirrored	17.2%	269
shifted	90.2%	1,119
gaussian	100.0%	2,231
dimmed	0.0%	18

Table 8: Detector quality against the number of components.

variance	d	false alarms	mean detection	ROC-AUC
80%	44	2.84%	67.0%	0.819
90%	87	3.12%	66.3%	0.806
95%	154	3.34%	65.0%	0.791
99%	331	3.67%	65.5%	0.786

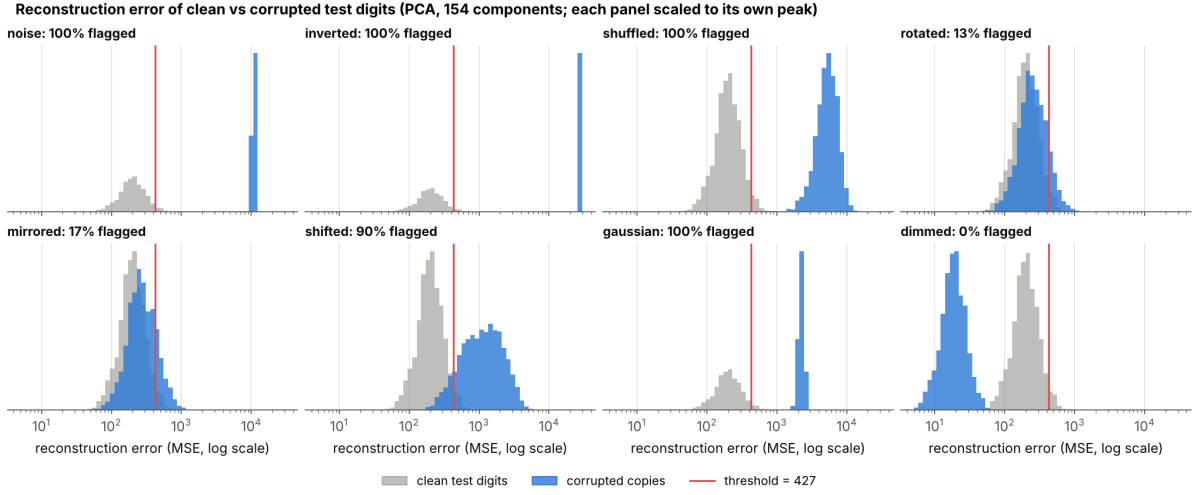


Figure 11: Reconstruction errors of clean and corrupted test digits, with the threshold.

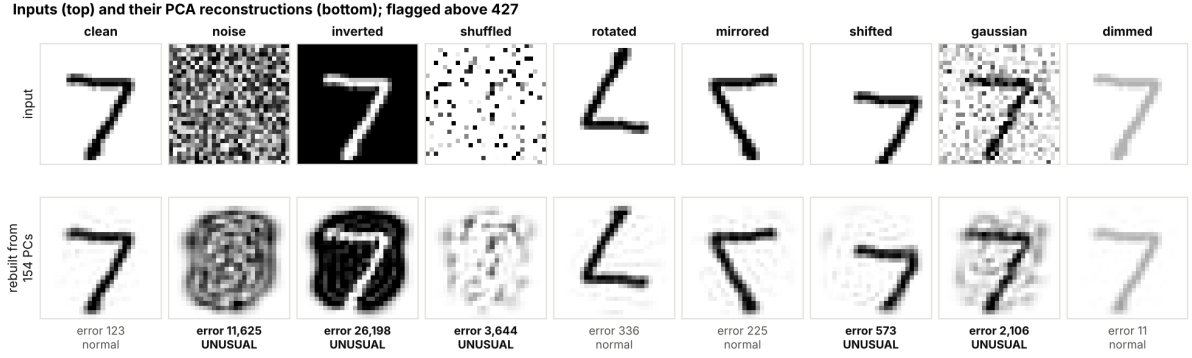


Figure 12: One test digit, its corruptions and their reconstructions from 154 components.

9 From notebook to service

Dimensionality Reduction MLOps: one config, one command (docker compose up)

Every box is a Docker Compose service, except the data source, the reports folder and the browser.

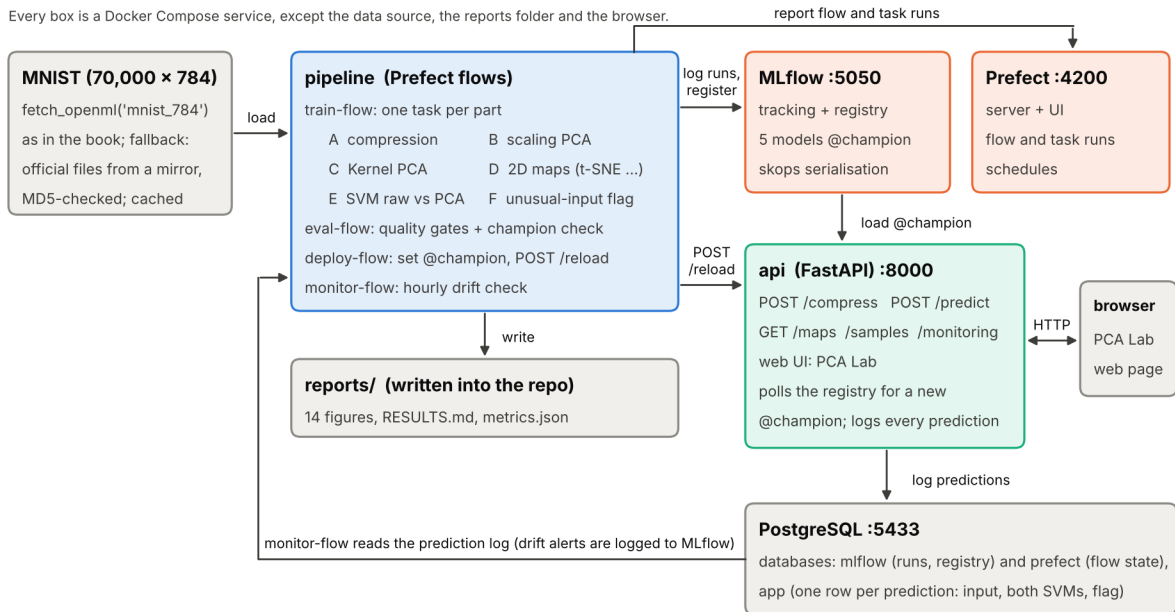


Figure 13: System architecture. All services start with `docker compose up`.

The experiments are packaged as a small production system (Figure 13). It follows the structure of a full-stack on-premises MLOps reference project, reduced to the parts this problem needs.

One configuration, one command. A YAML file describes the data split, every experiment, the quality gates and the deployment target. The command

```
python run_flow.py -config configs/full_flow_config.yaml
```

runs train → evaluate → deploy as Prefect flows. Inside Docker Compose the same command runs in the **pipeline** container. A quick configuration trains on a 10,000-image subset for continuous integration.

Tracking and registry. Each part (A–F) logs its parameters, metrics, tables and figures to its own nested MLflow run. Five models are registered with skops serialisation instead of pickle: the full PCA used for compression, the two SVMs, the reconstruction-error detector, and an *embedding atlas*. Before loading a model, the code checks that it asks to trust no custom types beyond an allow-list kept in the code itself. The atlas holds 2,000 points of every 2D map together with the fitted PCA, Kernel PCA and LLE reducers that can place new inputs.

Quality gates. The evaluation flow reloads the registered models and re-measures them on the test set. A candidate is promoted to the `@champion` alias only if every check passes:

- at most 200 components for 95% of the variance, and at least 94% of the test variance kept;
- both classifiers at least 95% accurate, with the PCA classifier at most 1 pp behind the raw-pixel classifier;

- at most 6% false alarms on clean digits, and at least 95% detection of noise, inverted and shuffled inputs;
- at least three maps able to place new points;
- no classifier more than 0.5 pp worse than the current champion.

Serving and monitoring. A FastAPI service loads the champions and offers several endpoints. `/compress` keeps any number of components and returns the rebuilt image. `/predict` returns both SVMs, the unusual-input flag and the input’s position on the maps. `/maps` and `/samples` feed the web interface. The service polls the registry for a new champion and logs every prediction to PostgreSQL. A scheduled monitoring flow raises an alert when the share of unusual inputs exceeds three times the expected 4%, when one digit dominates the predictions, or when the two classifiers disagree too often.

Operations. The pipeline exits with a non-zero code when a gate fails, and checks at start-up that it can write its reports. If the service cannot load a newly promoted set of models, the deploy flow restores the previous champion. The API loads each model by its resolved version number and ignores a half-finished promotion, so it never serves a mixed set. Every registered version is tagged with the MD5 of the data, the hash of the configuration and the git commit. Database passwords are removed from the configuration before it is logged. The scheduled drift check ignores demo and CI traffic.

Testing. `pytest` covers the data checks (including the MD5 rejection of a wrong file), the mathematical identities above, every experiment on small data, the gates, the monitoring rules and the API. An integration test runs the full Prefect and MLflow flow on an MNIST subset. GitHub Actions runs linting and the tests on Python 3.12 and 3.13, then builds the Docker images and smoke-tests the deployed service.

10 Limitations and next steps

- MNIST has a fixed split, so the results come from one test set of 10,000 images, given with confidence intervals. The same test set also drives the quality gates and the threshold sweep; a separate validation split would keep it untouched until the final report. Timings depend on the machine.
- The SVMs use default hyperparameters. Tuning C and γ for each representation, and an accuracy-versus- d curve, would separate the effect of compression from that of the kernel width more fully than the ablation above.
- The SVM is trained on 20,000 images to keep the pipeline fast. Training on all 60,000 would raise accuracy at a higher cost.
- The detector misses dimmed, mirrored and upside-down digits, and was tested on synthetic corruptions only. An intensity check, or a second score from a different model, would cover the first; real out-of-distribution images would test it properly.
- t-SNE was run with one perplexity (30) and one seed. Its map is a visual summary, not a model that can place new inputs.
- The deployed Kernel PCA uses scikit-learn’s default γ on MNIST. Tuning it for the maps, as was done on the Swiss roll, is a natural extension.

- Only methods from Chapter 8 of the book are used. Autoencoders, a neural approach to the same problem, are covered later in the book.

A Reproducing the results

- Full stack: `docker compose up -build`, then open `http://localhost:8000`.
- Local: `make install`, then `make train` (all 70,000 images) or `make quick`.
- Report: `python make_numbers.py`, then `pdflatex technical_report.tex` twice. Every number in this report is read from `reports/metrics.json`.